Arquitectura de Computadoras

Etapa N°1 del Proyecto

Fecha de Entrega: 21/04/2026

Comisión Número 19

Integrantes:
Duval Santino 152860,
García Damiani Pedro 152029,
Pelegrina Carrin Juan Manuel 151852

Versión de Logisim-Evolution: v4.1.0

Consigna: *"Etapa 1: Se desarrollará un sumador Carry Skip Adder de 16 bits implementado en el Trabajo práctico 3: TEMA A: Ejercicio 6 inciso a".*

En esta etapa se implementó un Carry-Skip Adder de 16 bits utilizando cuatro sumadores ripple de 4 bits cada uno. El diseño se realizó mediante el planteo de los subcircuitos necesarios para diseñar el Carry Skip Adder.
La jerarquía sería **Carry Skip Adder←Ripple Adder←Full Adder.**

FULL ADDER

El primer subcircuito es el Full Adder, que dados los bits A y B y un Carry de entrada C_in computa una suma y devuelve el bit S y un Carry de Salida C_out.

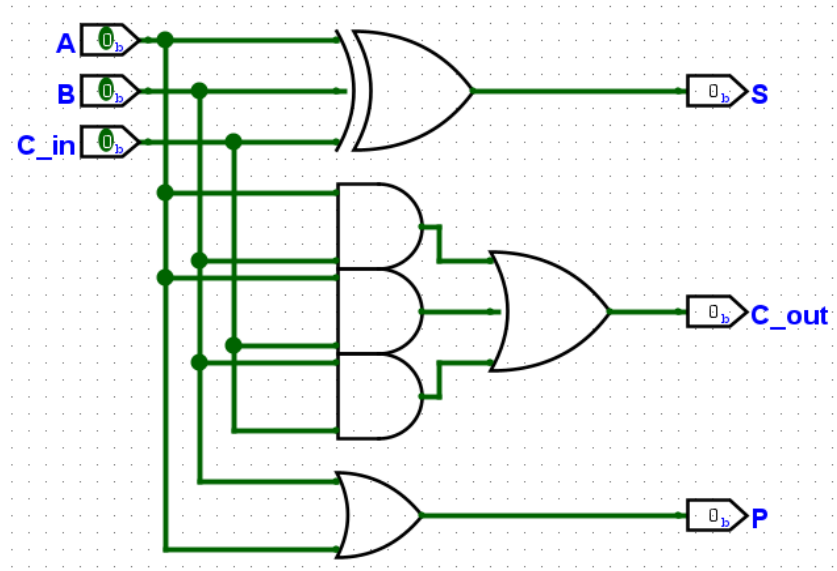
La ecuación lógica para el sumador en álgebra booleana es la siguiente:
S = A ⊕ B ⊕ C_in
C_out = A · B + P · C_in

Donde C_in es el bit de carry de entrada para calcular S, C_out es el carry que devuelve la suma en la i-ésima posición y P es la propagación y es de la forma **P = A + B.**

La tabla de verdad del circuito es la siguiente (donde C sería C_in en el circuito):

A	B	C	C_{OUT}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Y su diagrama lógico implementado en logisim con compuertas:



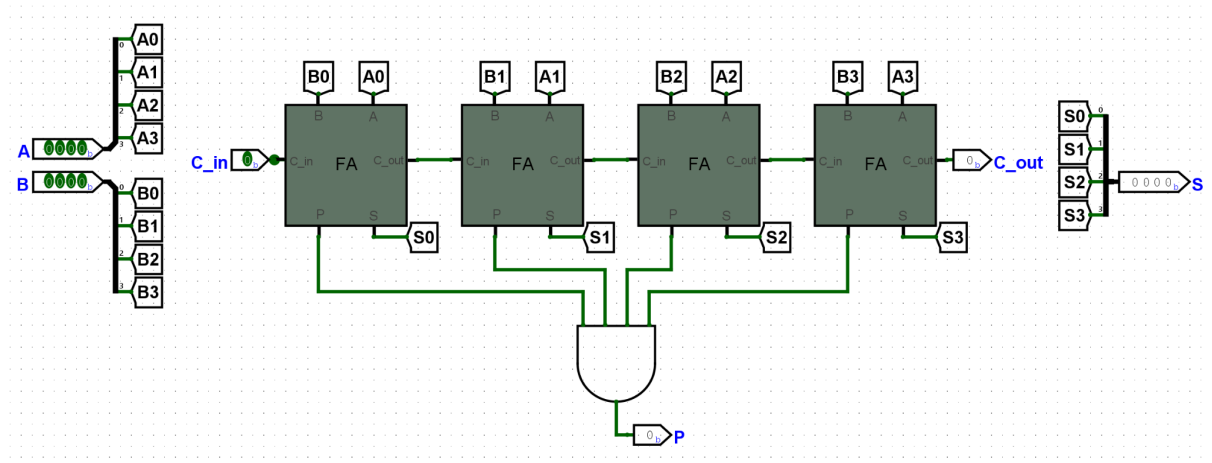
Este circuito cuenta con dos niveles de compuertas, 3 pines de entrada de 1 bit y 3 de salida de 1 bit .
El Xor que calcula **S = A ⊕ B ⊕ C_in** tiene 3 pines de entrada de 1 bit y está configurado en su comportamiento con el modo de imparidad de variables, que simula el comportamiento de varios (en este caso 2) Xor's concatenados. Ésta es una optimización por sobre concatenar dos Xor's para conseguir el mismo resultado, pues un Xor de tres entradas computa su función ligeramente más lento que uno de dos, mientras que usar 2 de 2 introduce un nivel de compuertas innecesario al cálculo que duplica el tiempo de cómputo con respecto a un Xor de dos entradas.

RIPPLE ADDER

Continuamos con los subcircuitos diseñando un Ripple Adder. Los Ripple Adders consiguen computar el resultado de una suma de n bits a través de n Full Adders conectados en cascada. Para esto, toma por entrada 2 números binarios no signados de n bits y 1 bit como Carry de entrada y retorna 1 número binario no signado de n bits y 1 bit como Carry de salida.

Los datos de los bits a los cuales los Full Adders han de computar su suma son recibidos por estos como inputs al Ripple Adder. Una ventaja de esta manera de introducir los datos es que las propagaciones de todos los Full Adders pueden ser calculadas de manera sincrónica en el tiempo que tarda un nivel de compuertas.

El diagrama lógico que representa nuestro diseño de un Ripple Adder de 4 bits es el siguiente:



Las entradas A y B del Adder son pines de 4 bits cada una, de modo de poder sumar los 4 bits requeridos por las especificaciones de la consigna. Éstas entradas luego son divididas en sus 4 valores y transportadas con Tunnels a las entradas de los Full Adders para procesar las sumas correspondientes.

La limitación infranqueable del diseño es el hecho de que para yo poder calcular la suma Si requiero primero el bit de carry Ci que debe ser calculado en un Full Adder previo (a menos que sea el primer Full Adder) y por lo tanto Si no se puede computar de forma sincrónica.
Si el retardo de una compuerta lógica es **t**, entonces **2t** es el retardo de un Full Adder, y por lo tanto, este Ripple Adder de 4 bits tiene un retardo de **2t*4 = 8t**.

El pin de entrada C_in ingresa un carry inicial para comenzar el proceso. Luego, los cuatro resultados S0, S1, S2 y S3 de las sumas son combinados para devolver una salida de 4 bits de nombre S y C_out.

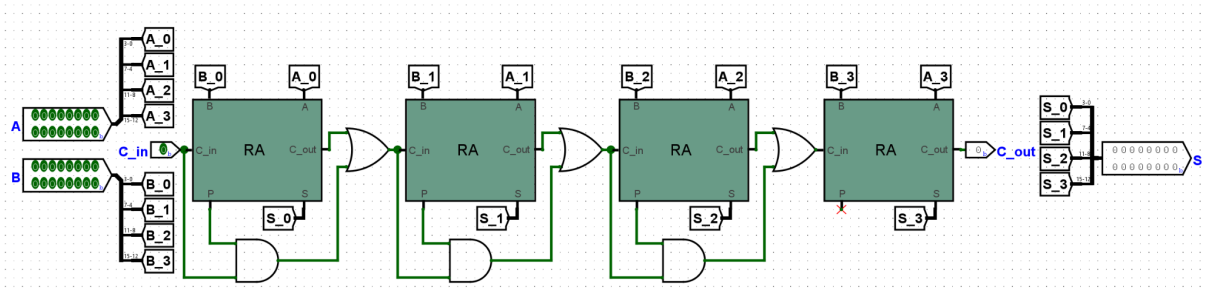
Por último, este Ripple Adder calcula la propagación del Carry de entrada y lo retorna mediante la salida **P = (A0+B0)·(A1+B1)·(A2+B2)·(A3+B3)**

CARRY SKIP ADDER

El circuito principal a diseñar para la etapa primera del proyecto es el Carry Skip Adder. Este circuito se construye al conectar en cascada varios Ripple Adders y logra optimizar el retardo de hacer sumas al utilizar la propagación combinada con el carry de entrada de cada Ripple Adder para poder predecir si este va a tener un carry de salida, así pudiendo comenzar el procedimiento de suma del próximo Ripple Adder sin requerir que todas las sumas dentro de este Ripple Adder sea terminada de antemano.

Si yo tengo una propagación del carry a lo largo de todo un RA (Ripple Adder) y además ese RA tuvo un carry de entrada estoy seguro de que, independientemente del resultado del cálculo, voy a tener un carry de salida.

La implementación del diseño en forma de diagrama lógico:



La entrada C_in representa el carry de entrada y las entradas A y B de 16 bits representan los dos números que van a ser sumados y son divididos en 4 buses de 4 bits cada uno para ser ingresados en los RA de 4 bits correspondientes.

sea ΔG el retardo de una compuerta lógica, k la cantidad de bits de los operandos que se suman en ripple y n la cantidad de bits por operando a sumar

El tiempo de suma T de un Carry Skip Adder está dado por ΔG
T = t_ripple en k bits + (n/k − 2)(t_AND + t_OR) + t_ripple en k bits

$T = \Delta G\ (4k + 2n/k - 4)$

En este caso en particular nuestro ripples son de k = 4 bits y nuestra cantidad de bits totales es de n = 16 bits. luego $T = \Delta G(16+8-4) = 20\ \Delta G$. Además el T de tener simplemente RA's sin el Carry Skip Adder sería de $4*2*4 = 32\ \Delta G$. O sea que este diseño plantea una gran optimización en prácticamente todos los casos menos el peor de todos.

Por último los resultados de cada RA son combinados y salen por el pin S, además del C_out.

Este circuito particularmente no está diseñado para ser concatenado con otros del mismo tipo sin perder una parte de su eficiencia en el camino, pues la comparación del carry de entrada y propagación del último RA es salteada en pos de ahorrar en compuertas lógicas.